

5-MARK QUESTIONS (LONG ANSWERS)

1. Compare Scrum and Extreme Programming (XP). Recommend the most suitable framework.

Scrum is an Agile framework mainly focused on project management. It works in fixed-length iterations called sprints. Scrum emphasizes roles like Product Owner, Scrum Master, and Development Team, along with ceremonies such as Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective.

Extreme Programming (XP) focuses more on engineering practices. It emphasizes continuous testing, pair programming, test-driven development (TDD), continuous integration, and simple design.

Comparison: - Scrum focuses on *management and delivery*, while XP focuses on *technical excellence*. - Scrum does not define coding practices; XP strictly defines them. - XP is suitable when requirements change frequently and high-quality code is required.

Recommendation: For projects with rapidly changing requirements and strong quality needs, **XP is more suitable**, while Scrum is better for managing complex projects.

2. Explain SOLID principles and their importance in Agile projects.

SOLID principles are object-oriented design principles that improve software maintainability and scalability.

- **Single Responsibility Principle (SRP):** A class should have only one responsibility.
- **Open Closed Principle (OCP):** Software entities should be open for extension but closed for modification.
- **Liskov Substitution Principle (LSP):** Subclasses should be substitutable for their base classes.
- **Interface Segregation Principle (ISP):** Clients should not be forced to depend on unused interfaces.
- **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules.

In Agile projects, SOLID principles help in easy refactoring, faster changes, and better scalability, especially in cloud-based systems.

3. Explain Test-Driven Development (TDD) with example.

Test-Driven Development (TDD) is an Agile practice where test cases are written before the actual code.

TDD Cycle: 1. Write a failing test (Red) 2. Write minimum code to pass the test (Green) 3. Improve the code (Refactor)

Benefits: - Improves code quality - Reduces defects - Encourages better design

Example: For a login feature, write a test to validate username and password first, then write code to pass the test.

4. Explain Continuous Integration (CI) and its importance.

Continuous Integration is an Agile practice where developers frequently integrate code into a shared repository.

Importance of CI: - Early detection of bugs - Improves collaboration - Faster releases - Reduces integration risks

Common Tools: Jenkins, Git, GitHub Actions

CI is essential in DevOps environments for automated build and testing.

5. Explain DevOps lifecycle with tools.

DevOps is a practice that combines development and operations.

DevOps Lifecycle Phases: - Plan: Jira - Code: Git - Build: Maven - Test: Selenium - Release: Jenkins - Deploy: Docker, Kubernetes - Monitor: Nagios

DevOps improves deployment speed and software reliability.

6. Explain Function Point Analysis (FPA).

Function Point Analysis is a technique used to estimate software size based on functionality.

Components: - External Inputs (EI) - External Outputs (EO) - External Inquiries (EQ) - Internal Logical Files (ILF) - External Interface Files (EIF)

Steps: 1. Calculate Unadjusted Function Points (UFP) 2. Calculate Complexity Adjustment Factor (CAF) 3. Final FP = UFP × CAF

7. Explain Basic COCOMO model.

COCOMO is a software cost estimation model.

Basic COCOMO Formula: - Effort = $a \times (\text{KLOC})^b$ - Time = $c \times (\text{Effort})^d$

Project Types: - Organic - Semi-Detached - Embedded

It helps in estimating development effort and time.

3-MARK QUESTIONS (SHORT ANSWERS)

Agile Manifesto Values

- Individuals and interactions over processes and tools
 - Working software over documentation
 - Customer collaboration over contract negotiation
 - Responding to change over following a plan
-

Scrum Roles

- Product Owner: Manages product backlog
 - Scrum Master: Facilitates Scrum process
 - Development Team: Builds the product
-

Alpha vs Beta Testing

- Alpha: Internal testing
 - Beta: User-side testing
 - Purpose: Identify defects before release
-

Error vs Bug vs Failure

- Error: Human mistake
 - Bug: Defect in code
 - Failure: System not working as expected
-

Cyclomatic Complexity

Cyclomatic Complexity measures the complexity of a program.

Formula: $V(G) = E - N + 2$

What is Refactoring?

Refactoring is improving code structure without changing functionality.

Benefits of DevOps

- Faster deployment
- Better collaboration
- Improved quality